Security Assessment

Final Report

Royco Day

August 2026

Prepared for Royco

certora

ROYCO

Table of Contents

certora ROYCO

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Royco Day	https://github.com/roycoprotocol/royco-day	539418f (initial) df23c29 (final)	EVM

Project Overview

This document describes the security review of **Royco Day**. The work was undertaken from **July 20th, 2026** to **August 7th, 2026**.

The following contract list is included in our scope:

- src/*

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Note: *Some findings identified during this audit were introduced by code changes made after the audit began and were not present in the initial audit commit. As a result, the report covers both issues in the initial codebase and issues arising from mid-audit changes reviewed as part of the updated scope.*

Protocol Overview

Royco Day is a decentralized finance protocol that restructures yield-bearing assets into three risk-adjusted tranches: senior, junior, and liquidity provider. Senior participants receive downside protection and guaranteed secondary liquidity, junior participants supply first-loss capital in exchange for a risk premium, and liquidity providers supply market-making capital in return for additional yield.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Royco Day separates a yield-bearing collateral position into senior (ST), junior (JT), and liquidity-provider (LPT) tranches. The Kernel routes deposits, redemptions, and Balancer V3 liquidity operations; the Accountant maintains the loss waterfall, premiums, fees, and market state; YDMs determine yield allocation; and the Entry Point provides delayed, oracle-gated settlement. Markets are deployed through approved factory templates and governed through a shared AccessManager. External trust surfaces include collateral and LP oracles, Chainlink feeds, ERC-4626/Idle/Makina integrations, Balancer V3, sanctions screening, and protocol periphery.

The principal invariants are that collateral NAV equals ST plus JT effective NAV, internal ledgers remain token-backed, losses are absorbed junior-first, recoveries are allocated independently of sync cadence, and fee or premium share minting does not create NAV or weaken coverage. Operations must preserve minimum coverage and liquidity, liquidation bonuses must not worsen remaining coverage, previews must match execution, and queued requests must respect delay, oracle-update, escrow, and yield-forfeiture rules. Users, request executors, pool traders, and enabled templates were treated as potentially adversarial. Governance multisigs, configured oracle sources, approved implementations, and external protocol correctness remain trusted assumptions; compromise of privileged roles can change parameters, oracles, implementations, or emergency state.

The review considered privilege escalation, deployment and upgrade misconfiguration, malicious-template behavior, reentrancy and callback surfaces, rounding and share inflation, stale or manipulated pricing, path-dependent accounting, partial-fill manipulation, liquidation-bonus arbitrage, abnormal token behavior, and Balancer swaps, donations, joins, exits, rate updates, and fee interactions. Manual review covered the factory and role graph, tranches, Kernel, Accountant, Entry Point, YDMs, oracle adapters and clocks, blacklist logic, Balancer venue, and deployment and upgrade tooling. Concrete, fuzz, fork, lifecycle, and stateful invariant tests exercised initialization and access control, accounting under gains and losses, fixed-term and liquidation transitions, deposit and redemption boundaries, preview parity, queued settlement, multi-asset flows, oracle freshness, non-standard tokens, external pool drift, and deployment wiring. Stateful campaigns checked NAV conservation, ledger backing, state-machine consistency, YDM bounds, oracle-clock behavior, and reduction to a senior/junior-only market.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	1	1	1
Medium	11	11	10
Low	14	14	9
Informational	2	2	1
Total	28	28	21

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
H-01	One-wei bootstrap rate lets an attacker steal the first ST liquidity deposit	High	Fixed
M-01	Full asset recovery does not make JT whole	Medium	Fixed
M-02	Indefinitely executable requests still allow oracle update front-running	Medium	Fixed
M-03	EntryPoint mixes post-sync NAV with stale tranche supply	Medium	Fixed
M-04	LT redemption yield forfeiture ignores idle liquidity-premium value	Medium	Fixed
M-05	Minimum coverage updates can create unearned liquidation bonuses	Medium	Fixed
M-06	LPT deposit references mix pre-sync BPT value with post-sync accounting	Medium	Fixed
M-07	Permissionless dust initialization can grief the Balancer liquidity venue	Medium	Fixed
M-08	Makina share-price oracle reports fresh Chainlink timestamps for stale cached AUM	Medium	Fixed

M-09	Attacker can trigger Balancer round-trip fee on EP multi-asset redemptions	Medium	Fixed
M-10	Post-operation reinvestment misprices idle liquidity-premium shares	Medium	Fixed
M-11	Asynchronous requests lack slippage protection	Medium	Acknowledged
L-01	IdleCDOTranchePriceOracle ignores the Chainlink feed timestamp	Low	Fixed
L-02	New losses inherit the original fixed-term expiry	Low	Acknowledged
L-03	Hookless Balancer interactions leave fee-adjusted LPT NAV stale	Low	Acknowledged
L-04	Makina recovery mode does not stop Royco from pricing DUSD	Low	Acknowledged
L-05	Sync cadence changes impermanent-loss recovery priority	Low	Fixed
L-06	Whitelisted shareholders can bypass EntryPoint delays and oracle gating	Low	Fixed
L-07	Executor controls the LPT redemption asset composition	Low	Fixed
L-08	Idle CDO composite prices cannot enforce source-specific staleness limits	Low	Fixed

L-09	Balancer exit fee after hook-triggered reinvestment	Low	Fixed
L-10	The liquidation threshold cannot be updated after the junior buffer is exhausted	Low	Fixed
L-11	Multi-asset flows reject partial liquidity healing	Low	Acknowledged
L-12	Chainlink updates can mask stale ERC4626 share prices	Low	Fixed
L-13	Deprecated answeredInRound check limits oracle compatibility	Low	Fixed
L-14	previewRedeem might be unsafe for generalized ERC4626 pricing	Low	Acknowledged
I-01	Risk parameters cannot be changed while the market is paused	Informational	Acknowledged
I-02	In-kind liquidity redemptions revert for holders without the senior role	Informational	Fixed

High Severity Issues

H-01: One-wei bootstrap rate lets an attacker steal the first ST liquidity deposit

Severity: High	Impact: High	Likelihood: Medium
Files: src/kernels/base/liquidity-venue/balancer-v3/BalancerV3LiquidityVenue.sol	Status: Fixed	

Description:

During a multi-asset LPT deposit, Royco cached the ST share rate during the initial accounting sync. It then minted ST shares from the collateral contribution and added them with the quote assets to Balancer without refreshing the cached rate.

This assumes that minting ST shares cannot change their rate. While normally true, the assumption fails during both the initial bootstrap and a clamp-limited mint.

In a fresh market, ST supply and effective NAV are zero, so the cached rate is zero. `getRate` replaces this with 1 to avoid returning a rate Balancer rejects:

```
return (rate == 0 ? 1 : rate);
```

Because Balancer rates use WAD precision, this values the ST leg at one quintillionth of its intended bootstrap value. Meanwhile, the first ST shares are minted 1:1 and become fully backed.

An attacker can exploit this by initializing the public Balancer pool with quote assets and retaining the resulting BPT. When a victim makes the first collateral-bearing multi-asset deposit, Balancer values the victim's newly minted ST shares using the stale one-wei rate. The victim therefore receives BPT mainly for their quote contribution. After the transaction, the live ST rate becomes approximately WAD, allowing the attacker to remove liquidity and capture most of the victim's ST shares.

The same root cause causes an additional loss when the mint-dilution clamp binds. A clamped mint issues fewer ST shares than fair pricing would produce, while the collateral still increases ST

effective NAV by its full value. The post-mint ST rate is therefore higher than the cached pre-deposit rate. Balancer nevertheless uses the lower cached rate and mints fewer BPT than the deposited shares' settled value warrants. This transfers additional value to existing BPT holders.

The protocol preview does not protect clamped depositors because it calculates the same limited share amount and simulates the Balancer add using the same stale rate. A minimum BPT amount derived from the preview therefore accepts the underpriced execution.

The impact is that an attacker can steal nearly all collateral represented by the first ST liquidity contribution. In an already active market, a depositor whose ST mint is clamped can also lose the difference between the minted shares' pre-deposit and post-deposit valuations to existing BPT holders.

Recommendations:

Compose the multi-asset flow from two complete in-kind deposits. First execute the ST deposit, including its post-operation sync, and unconditionally refresh the cached ST rate from the settled ST effective NAV and supply. Then add liquidity to Balancer and complete the LPT in-kind deposit. Previews should simulate and revert the same sequence.

Customer Response:

Fixed in fc3732f.

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: Full asset recovery does not make JT whole

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/accountant/RoycoDayAccountant.sol	Status: Fixed	

Description:

The accountant protects ST from losses by reducing JT effective NAV, recording the covered amount as JT coverage impermanent loss, and leaving ST effective NAV unchanged. This creates an ST cross-claim on JT raw NAV. During later synchronizations, `RoycoDayAccountant` attributes changes in JT raw NAV proportionally to that cross-claim.

The cross-claim participates in the appreciation of JT assets, but the recorded impermanent-loss obligation remains a fixed NAV amount. When the assets recover, the accountant uses ST-attributed gain to clear only that fixed obligation. Any appreciation attributed to ST through its cross-claim that exceeds the stored obligation is treated as new ST yield and enters the premium distribution logic, even though that appreciation arose from junior capital previously used to protect ST.

For example, assume ST and JT hold the same asset, the JT yield share is 10%, and all other premiums and fees are zero:

1. ST and JT begin with raw and effective NAVs of 1,000 and 300.

1. The asset price falls by 10%, reducing raw NAV to 900 and 270. JT absorbs its own 30 loss and covers ST's 100 loss. Effective NAV becomes 1,000 and 170, while the accountant records impermanent loss of 100.

1. At this checkpoint, ST's 1,000 effective NAV consists of its 900 raw NAV and a 100 cross-claim on JT's 270 raw NAV.

1. The asset price fully recovers, producing raw NAV gains of 100 for ST and 30 for JT. ST receives its full 100 gain plus 100/270 of JT's 30 gain through its cross-claim. ST-attributed gain is therefore 111.111, while JT is attributed the remaining 18.889.

1. The accountant uses 100 of ST's attributed gain to clear the stored impermanent loss. It classifies the remaining 11.111 as ST yield, so JT receives only its 10% yield share of 1.111 and ST retains 10.

1. Raw NAV has returned exactly to 1,000 and 300, but effective NAV ends at 1,010 and 290. The impermanent-loss balance is zero even though JT remains 10 below its starting NAV.

Consequently, a temporary price decline followed by a complete recovery permanently transfers value from JT to ST and weakens the junior coverage buffer. The stored impermanent loss reaches zero and the market can leave its fixed-term recovery state even though JT has not been made whole. Repeated price round trips can compound this transfer.

Recommendations:

Since ST and JT will be coinvested in the same asset, update the accountant so that an asset-price recovery first restores all effective NAV that JT lost while covering ST.

Customer Response:

Fixed in eabdafd.

Fix Review:

Fix confirmed.

M-02: Indefinitely executable requests still allow oracle update front-running

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/entrypoint/RoycoDayEntryPoint.sol	Status: Fixed	

Description:

`RoycoDayEntryPoint` delays deposit and redemption requests and, when an oracle clock is configured, requires the clock to report an update after the request was queued. However, `BaseRequest` stores only the time from which execution is allowed. It does not store an expiration time, and `_validateRequestExecution` enforces no upper execution bound:

The oracle check proves only that some update occurred after the request was created. Once the first such update occurs and the delay elapses, the condition remains satisfied indefinitely because every later clock timestamp is still greater than `queuedAtTimestamp`.

An attacker can therefore queue a self-executable redemption, wait until it becomes executable, and leave it pending. When an adverse oracle update appears in the public transaction pool, the attacker can front-run it by executing the old request. The previously observed oracle timestamp passes the gate, while the tranche's accounting sync still uses the old oracle value, allowing the attacker to exit before the loss is reflected. If an update is favorable instead, the attacker can leave the request pending or cancel it and recover the escrowed shares. Mature deposit requests can similarly be retained and selectively executed when stale pricing makes entry favorable.

The attacker can also periodically cancel and renew the redemption to keep a relatively recent request armed and reduce the yield forfeited upon execution. For example, at 5% APY, one week of accrued yield is approximately 0.1%. Executing a one-week-old request immediately before an oracle update that reduces the fair redemption value by more than 0.1% would therefore remain profitable despite the forfeiture. More frequent renewals can reduce this cost further. Cancellation itself returns the escrowed shares without forfeiture, so the attacker pays the yield cost only when choosing to exercise the stale-price exit.

As a result, users can maintain standing options around future oracle updates despite the intended request delay. They can avoid adverse repricing or enter before favorable repricing, shifting the resulting loss or dilution to the remaining tranche holders.

Recommendations:

Add a short execution window to `BaseRequest` and reject executions after it expires. The window should be selected together with the configured delay and oracle update cadence, and should ideally be short enough that an earlier oracle update cannot leave a request armed when the next update arrives.

Customer Response:

Fixed in 477bbe0.

Fix Review:

Fix confirmed.

M-03: EntryPoint mixes post-sync NAV with stale tranche supply

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/entrypoint/RoycoDayEntryPoint.sol	Status: Fixed	

Description:

`RoycoDayEntryPoint` snapshots the request-time price of asynchronous deposits and redemptions. A depositor may keep no more than the shares the deposit would have minted when requested, while a redeemer forfeits any value accrued by the queued shares before execution.

The `_depositSharesReference` and `_redemptionValueReference` helpers read the tranche supply directly from `totalSupply`, but obtain the corresponding NAV through `totalAssets`. The `totalAssets` call previews an accounting synchronization, so its returned NAV reflects reconciled profit and loss. That same synchronization can mint accrued protocol-fee shares to the senior and junior tranches, as well as liquidity-premium senior shares, but those simulated mints are absent from the separately read `totalSupply`. The helpers therefore combine post-sync NAV with pre-sync supply instead of using one coherent accounting state.

When shares are pending mint, `_depositSharesReference` understates how many shares the queued assets would mint. The eventual deposit performs a real synchronization and prices the mint against the increased post-sync supply. Even if no relevant price movement occurred during the queue, EntryPoint can treat the difference as favorable execution and retain valid user shares as protocol fee shares.

Conversely, `_redemptionValueReference` overstates the queued shares' value. If another operation realizes the pending synchronization after the request, the inflated request-time baseline can conceal genuine value accrued during the queue, allowing the redeemer to retain value that should have been forfeited. Additional unsynchronized mints before execution can further distort the execution-time comparison.

As a result, ordinary pending fee or liquidity-premium accrual can cause depositors to lose shares without an adverse price movement and can cause the protocol to miscalculate the value forfeited from redemptions.

Recommendations:

Call the kernel's `previewSyncTrancheAccounting` function once and use both the NAV or asset claims and `totalTrancheShares` returned by that preview.

Customer Response:

Fixed in e011b5a.

Fix Review:

Fix confirmed.

M-04: LT redemption yield forfeiture ignores idle liquidity-premium value

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/entrypoint/RoycoDayEntryPoint.sol src/tranches/base/RoycoVaultTranche.sol	Status: Fixed	

Description:

`RoycoDayEntryPoint` snapshots the NAV of shares placed in a redemption request and compares it with their NAV at execution. If the execution-time NAV is higher, `_redeemWithYieldForfeiture` retains the corresponding LT shares as a protocol fee so that the redeemer cannot capture value accrued while the request was queued.

Both snapshots are obtained from `convertToAssets`. For LT, however, `RoycoVaultTranche` deliberately removes idle ST-share claims from that conversion and replaces effective NAV with raw NAV. Effective LT NAV consists of both the deployed market-making inventory and the value of ST shares received as liquidity premium, so the EntryPoint's reference omits part of the value already backing the queued shares.

This makes the fee depend on the form in which LT value is held rather than on whether value actually accrued. For example:

1. A redeemer queues LT shares with a pro rata claim on 100 of raw NAV and 10 of idle ST-share value. Their total effective value is 110, but the request records only the 100 of raw NAV.

1. During the queue, the existing idle ST shares are reinvested and become 10 of additional raw NAV. The total effective value remains 110, assuming no reinvestment loss.

1. At execution, `convertToAssets` reports 110 of raw NAV. EntryPoint compares it with the request-time value of 100 and incorrectly treats the existing 10 as newly accrued yield, causing the redeemer to forfeit shares despite earning no new value.

1. In the opposite case, assume the shares initially have 100 of raw NAV and no idle ST-share value. If 10 of new liquidity premium accrues as idle ST shares during the queue, raw NAV remains 100. Both snapshots therefore report 100, and no yield fee is charged on the 10 of value that actually accrued.

As a result, routine liquidity-premium reinvestments can incorrectly transfer a redeemer's pre-existing value to the protocol, while newly accrued idle liquidity premiums can escape the intended redemption yield forfeiture. The discrepancy can be as large as the value of idle ST shares moved into or added to the LT during the request delay.

Recommendations:

Calculate the redemption reference from the LT's complete effective NAV, including both its raw market-making inventory and its idle ST-share claims.

Customer Response:

Fixed in 477bbe0.

Fix Review:

Fix confirmed.

M-05: Minimum coverage updates can create unearned liquidation bonuses

Severity: Medium	Impact: High	Likelihood: Low
Files: src/accountant/RoycoDayAccountant.sol	Status: Fixed	

Description:

Coverage utilization is the product of total ST and JT raw NAV and the minimum coverage parameter, divided by JT effective NAV. Once it reaches the liquidation coverage utilization threshold, senior redeemers receive a self-liquidation bonus funded by JT effective NAV.

The `setMinCoverage` function uses `withSyncedAccounting`, but this modifier only synchronizes accounting before the setter body. The setter then verifies only that the new minimum coverage is below WAD and does not validate the market state produced by the new value:

```
    modifier withSyncedAccounting() {
        IRoycoDayKernel(KERNEL).syncTrancheAccounting();
        _;
    }

    function setMinCoverage(uint64 _minCoverageWAD) external restricted
withSyncedAccounting {
        require(_minCoverageWAD < WAD, INVALID_COVERAGE_CONFIG());
        $.minCoverageWAD = _minCoverageWAD;
    }
```

Because minimum coverage appears directly in the numerator of coverage utilization, increasing it can immediately push an otherwise solvent market above its liquidation threshold. A user who observes a scheduled administrator update can first deposit into ST, or otherwise move the market to the limit permitted by the old coverage requirement. When the update executes, coverage utilization jumps without any underlying loss. The user can then redeem ST and collect a self-liquidation bonus from JT.

As a result, a predictable administrative parameter update can be used to transfer value from honest JT liquidity providers to a prepared ST user. It can also place the market into an artificial liquidation state even though its asset values have not deteriorated.

Recommendations:

Synchronize accounting both before and after coverage-related parameter changes. Revert an update if post-update coverage utilization exceeds 100% and is greater than its pre-update value.

Customer Response:

Fixed in 6b6f5d6.

Fix Review:

Fix confirmed.

M-06: LPT deposit references mix pre-sync BPT value with post-sync accounting

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/entrypoint/RoycoDayEntryPoint.sol	Status: Fixed	

Description:

`RoycoDayEntryPoint` records `equivalentSharesAtRequestTime` when a deposit is requested. At execution, the depositor receives the lower of this reference and the shares actually minted, while any excess minted shares are retained as protocol fees.

For an LPT deposit, `_depositSharesReference` first converts the requested BPT amount to NAV using the pool's current TVL and BPT supply. Only afterward does it synchronize the market and calculate the share reference from the returned post-sync LPT claims and supply:

```
    NAV_UNIT depositValue = (_trancheType == TrancheType.LIQUIDITY_PROVIDER)
        ? IRoycoDayKernel(_kernel).convertLPTAssetsToValue(_assets)
        : IRoycoDayKernel(_kernel).convertCollateralAssetsToValue(_assets);

    (SyncedAccountingState memory state, AssetClaims memory trancheClaims,
uint256 totalTrancheShares) =
        IRoycoDayKernel(_kernel).syncTrancheAccountingFor(_trancheType);

    return ValuationLogic._convertToSharesUnclamped(
        depositValue, trancheClaims.nav, totalTrancheShares, Math.Rounding.Floor
    );
```

This ordering combines values from different accounting states. The synchronization can realize an accrued liquidity premium, mint senior tranche shares for the LPT, and reinvest those shares into the Balancer pool. A successful reinvestment changes the pool balances, TVL, and BPT supply used by `convertLPTAssetsToValue`. However, `depositValue` remains based on the pool state before reinvestment, while `trancheClaims` and `totalTrancheShares` reflect the state after it.

The discrepancy can cause an LPT depositor to lose shares as follows:

1. A user requests an LPT deposit after a liquidity premium has accrued.
1. `_depositSharesReference` values the requested BPT against the pre-sync pool.
1. The following synchronization mints and reinvests the premium, changing the BPT value, and the EntryPoint stores a share reference calculated from the stale deposit value and fresh LPT accounting.
1. When the request is executed without any intervening yield, the actual deposit uses the post-sync BPT value and can mint more shares than the understated reference.
1. `_depositWithShareForfeiture` treats the difference as queue-time appreciation and retains it as protocol fee shares.

As a result, routine liquidity-premium accounting can cause LPT depositors to forfeit part of their deposit even though the LPT generated no yield while the request was queued. The loss grows with the BPT repricing caused by the synchronization and violates the EntryPoint's guarantee that only favorable movement during the queue is forfeited.

Recommendations:

Synchronize the market before converting the requested LPT assets to value. The BPT conversion, LPT NAV, and LPT share supply used for `equivalentSharesAtRequestTime` should all come from the same post-sync accounting state.

Customer Response:

Fixed in d314a4c.

Fix Review:

Fix confirmed.

M-07: Permissionless dust initialization can grief the Balancer liquidity venue

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/kernels/base/liquidity-venue/balancer-v3/hooks/RoycoDayBalancerV3Hooks.sol	Status: Fixed	

Description:

The deployment flow creates and registers the Gyro ECLP pool but deliberately leaves it uninitialized. `RoycoDayBalancerV3MarketDeploymentTemplate` even requires the pool to have zero BPT supply when the market is wired, and no deployment step seeds it afterward. At the same time, both versions of the Royco Balancer hook disable the before-initialize and after-initialize callbacks. The canonical Balancer Router can therefore initialize the completed pool without any Royco authorization or validation.

Once an external party initializes the pool, `BalancerV3VenueLogic` permanently selects the unbalanced-add path for every Royco deposit:

```
if (_immutables.vault.isPoolInitialized(_immutables.lptAsset)) {

    _immutables.vault.addLiquidity(... AddLiquidityKind.UNBALANCED ...);

} else {

    _immutables.vault.initialize(...);

}
```

An attacker can exploit this during market bootstrapping as follows:

1. After the market is wired but before its first legitimate LPT deposit, initialize the pool through the public Balancer Router with no ST shares and only one raw unit of USDC. The USDC scaling to 18 decimals produces enough invariant to exceed Balancer's minimum BPT supply and permanently marks the pool initialized.

1. A later quote-only Royco deposit enters the unbalanced-add branch. Balancer computes each new balance as current balance plus input minus one to undo prior rounding. For the ST leg, both the stored balance and input are zero, so the calculation becomes zero plus zero minus one and reverts.

1. A normal ST-backed deposit also enters the unbalanced-add branch. Because the attacker's genesis invariant is extremely small, any meaningful deposit increases it by more than Gyro ECLP's maximum invariant ratio of 500% and reverts with the invariant-growth bound.

The initialized flag cannot be returned to the genesis state. As a result, an attacker can cheaply prevent normal multi-asset LPT deposits and quote-only deposits. Since positive senior deposits depend on existing LPT depth, the attack can prevent the entire market from bootstrapping until the pool is operationally recovered through carefully staged additions or the market is redeployed.

Recommendations:

Initialize the pool atomically as part of market deployment with adequately sized two-sided liquidity at the intended ECLP ratio.

Customer Response:

Fixed in a3fd452.

Fix Review:

Fix confirmed.

M-08: Makina share-price oracle reports fresh Chainlink timestamps for stale cached AUM

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/oracle/MakinaSharePriceOracle.sol	Status: Fixed	

Description:

`MakinaSharePriceOracle` calculates the value of a Makina share by multiplying the machine's `convertToAssets` conversion rate by the accounting asset's Chainlink price:

```
price = toNAVUnits(
    _getCollateralToReferenceAssetConversionRateWAD().mulDiv(
        uint256(answer),
        ORACLE_PRICE_PRECISION,
        Math.Rounding.Floor
    )
);
updatedAt = feedUpdatedAt;
```

However, `convertToAssets` is not a live valuation. Makina calculates it using its cached `lastTotalAum` and the machine share supply. Investment gains and losses are incorporated into `lastTotalAum` only when Makina performs global accounting through `updateTotalAum`, which also advances `lastGlobalAccountingTime`.

Royco's abridged `IMachine` interface does not expose `lastGlobalAccountingTime`, and `MakinaSharePriceOracle` inherits `getPrice`, `poke`, and `previewPoke` from `ChainlinkPriceOracleBase`. Consequently, all three functions report only the Chainlink feed timestamp:

```
function poke() public virtual override(IRoycoPriceOracle) returns (uint256
updatedAt) {
    (, updatedAt) = getPrice();
```

```
}

function previewPoke() public view virtual override(IRoycoPriceOracle) returns
(uint256 updatedAt) {
        (, updatedAt) = getPrice();
}
```

This incorrectly treats a new Chainlink round as an update to the entire composed Makina share price, even when the cached AUM component has not changed.

This affects both consumers of the oracle timestamp:

- `RoycoDayKernel` accepts the composed price when its reported timestamp is within the configured staleness threshold. A recent Chainlink round can therefore make an arbitrarily old Makina AUM conversion appear current.
- When `gateByOracleUpdate` is enabled, `RoycoDayEntryPoint` permits execution only when `poke` returns a timestamp strictly later than the request's `queuedAtTimestamp`. A user can queue a deposit or redemption after the latest Makina accounting update, wait for a new Chainlink round, and execute before Makina performs another global accounting update. The Chainlink timestamp passes the gate even though no new Makina valuation has been incorporated.

If upcoming Makina accounting updates are predictable, a user can execute immediately before a favorable AUM change is incorporated. This defeats the intended post-request update requirement and may allow deposits or redemptions to execute using a stale share conversion, transferring value between the user and other tranche holders.

Recommendations:

Extend `IMachine` to expose `lastGlobalAccountingTime` and override `getPrice` in `MakinaSharePriceOracle` so the report uses the oldest timestamp among its price components.

Because the inherited `poke` and `previewPoke` call `getPrice`, this override ensures that a Chainlink update alone cannot advance the oracle clock while Makina's cached AUM remains stale.

Customer Response:

Fixed in 86af1b4.

Fix Review:

Fix confirmed.

M-09: Attacker can trigger Balancer round-trip fee on EP multi-asset redemptions

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/kernels/base/liquidity-venue/balancer-v3/BalancerV3LiquidityVenue.sol	Status: Fixed	

Description:

Balancer tracks whether liquidity has been added to a pool during the current Vault unlock session. If the same session later performs a proportional removal, Balancer deducts the pool's static swap fee from every token amount returned by the removal. The add-liquidity flag is set even when the addition has zero value.

`BalancerV3LiquidityVenue.removeLiquidity` calls `Vault.unlock` without first checking whether another party has already unlocked the Vault. When the Vault is already unlocked, this nested call shares the outer unlock's session, including its add-liquidity flag. An attacker can therefore impose the fee on another user's executable EntryPoint redemption as follows:

1. Open an outer Balancer Vault unlock and perform a zero-value add to the Royco market's Balancer pool, setting the session's add-liquidity flag.

1. During the same callback, execute the victim's queued EntryPoint multi-asset redemption.

1. The EntryPoint calls `redeemMultiAsset` with zero minimum amounts. The redemption reaches `BalancerV3LiquidityVenue.removeLiquidity` and performs its proportional removal inside a nested unlock that inherits the attacker's session.

1. Balancer sees that liquidity was added to the pool in the same session and deducts the static swap fee from the victim's senior-share and quote-asset outputs. Since the EntryPoint supplied zero minimum amounts, the fee does not cause the redemption to revert.

As a result, an attacker can force a victim's queued multi-asset redemption to settle for less collateral and quote than an independent redemption would return. The loss is proportional to

the redeemed amount and the pool's static swap fee, while the attacker only needs a zero-value liquidity operation and does not need to own the victim's shares.

Recommendations:

Require the Balancer Vault to be locked before every Royco venue operation that opens an unlock. In particular, `addLiquidity`, `removeLiquidity`, and liquidity-premium reinvestment should revert if `Vault.isUnlocked` reports an existing session.

Customer Response:

Fixed in 7e55111.

Fix Review:

Fix confirmed.

M-10: Post-operation reinvestment misprices idle liquidity-premium shares

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/libraries/logic/AccountingSyncLogic.sol src/tranches/base/RoycoVaultTranche.sol src/accountant/RoycoDayAccountant.sol src/libraries/logic/liquidity-venue/BalancerV3VenueLogic.sol	Status: Fixed	

Description:

Liquidity-premium senior shares held by the kernel are reinvested into the Balancer pool. The reinvestment's minimum-BPT-output check must value those shares at the same senior share rate used by the pool.

In the affected implementation, reinvestment ran during the post-operation sync and valued the idle shares using the post-operation senior effective NAV divided by the current senior `totalSupply`. These values did not describe the same accounting state. A senior deposit had already increased NAV, but its shares were not minted until after the kernel returned. Conversely, a senior redemption had already reduced NAV, but the redeemed shares were not burned until after the kernel returned.

Meanwhile, the Balancer rate provider continued to use the senior rate cached by the pre-operation sync. The reinvestment floor and the pool therefore valued the same senior shares at different rates.

For a senior deposit with value `D`, pre-deposit senior NAV `N`, and reinvestment tolerance `t`, the floor was inflated by `1 + D/N`. Even assuming no actual venue slippage, reinvestment could not satisfy its floor when:

```
D/N > t/(1 - t).
```

With the shipped 10-basis-point tolerance, a deposit exceeding approximately 0.1001% of pre-deposit senior NAV was sufficient to make the attempt fail. The failure was intentionally skipped and emitted `LiquidityPremiumReinvestmentFailed`, leaving the premium shares idle for a later retry.

For a senior redemption, the error operated in the opposite direction and weakened the slippage check. Ignoring rounding and any self-liquidation bonus, redeeming a fraction f of senior supply changed the effective tolerance to:

```
1 - (1 - f)(1 - t).
```

Thus, redeeming 1% of senior supply changed a configured 10-basis-point tolerance into approximately 109.9 basis points.

Recommendations:

Mint or burn the operation's senior shares inside the kernel before the post-operation sync and reinvestment. This ensures the post-operation senior NAV and `totalSupply` represent the same settled state when calculating the reinvestment floor.

Customer Response:

Fixed in 67d629a.

Fix Review:

Fix confirmed.

M-11: Asynchronous requests lack slippage protection

Severity: Medium	Impact: High	Likelihood: Low
Files: src/entrypoint/RoycoDayEntryPoint.sol	Status: Acknowledged	

Description:

`RoycoDayEntryPoint` allows asynchronous deposit and redemption requests to be executed by third parties when the user does not select the self-execution sentinel. However, requests contain no user-defined minimum execution value or maximum acceptable loss. Deposits are the primary concern because an adverse price movement can cause the forfeiture mechanism to transfer part of the depositor's value directly to the protocol.

For a deposit, let `V0` and `P0` be the deposited assets' value and tranche share price at request time. The request stores approximately `V0 / P0` shares as its reference. At execution, if the assets are worth `V1` and the share price is `P1`, the deposit mints approximately `V1 / P1` shares. When this exceeds the stored reference, the user is capped at `V0 / P0` shares and the protocol receives the excess:

```
(V1 / P1) - (V0 / P0)`
```

There is no limit on how much value may be forfeited.

The issue can happen as follows:

1. A perpetual market has $9,000 ST and $1,000 JT backed by $10,000 of collateral. There are 1,000 JT shares worth $1 each.

2. A user queues 100 collateral tokens worth $100, fixing a reference of approximately 100 JT shares and enabling third-party execution.

3. Before execution, the oracle falls 5%. Collateral NAV falls to $9,500, and the $500 loss is absorbed by JT. JT NAV falls from $1,000 to $500, halving its share price to $0.50.

4. A keeper executes the request. The escrowed collateral is now worth $95 and mints approximately 190 JT shares.

5. The user is capped at 100 JT shares worth $50, while the remaining 90 shares worth $45 are forfeited to the protocol.

Cancellation would return collateral worth $95. Execution instead leaves the user with $50 before any executor bonus, crystallizing an additional $45 loss. This is a 47.4% loss compared with cancellation and a 50% loss relative to the original request value.

Malicious intent is unnecessary. A keeper may execute every eligible request normally, or the oracle price may change between transaction submission and inclusion. Disabling third-party execution avoids this path only by removing the keeper functionality; it does not protect users who enable it intentionally or accidentally.

Redemptions have a similar, though less significant, timing risk: they may settle after a sudden decline without checking a user-defined minimum value. Therefore, asynchronous requests should include user-defined execution bounds, with deposits receiving particular attention.

Recommendations:

Add user-defined execution protection to deposit and redemption requests. Deposits should revert when the execution-time value of the shares retained for the user falls below a specified minimum, or when the forfeiture exceeds a specified maximum. Redemptions should revert when the post-forfeiture claims fall below a specified minimum value.

Customer Response:

Acknowledged.

Low Severity Issues

L-01: IdleCDOTranchePriceOracle ignores the Chainlink feed timestamp

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/IdleCDOTranchePriceOracle.sol	Status: Fixed	

Description:

`IdleCDOTranchePriceOracle` composes an Idle CDO tranche's virtual price with the price returned by a Chainlink-compatible feed. `ChainlinkPriceOracleBase` returns both this composed price and the feed's update timestamp, but the override discards that timestamp and replaces it with the timestamp derived from the tranche virtual-price clock:

```
   function getPrice() public view override(ChainlinkPriceOracleBase) returns
(NAV_UNIT price, uint256 updatedAt) {
        (price,) = ChainlinkPriceOracleBase.getPrice();
        updatedAt = previewPoke();
    }
```

As a result, the composed price can include an old Chainlink answer while reporting a recent Idle CDO virtual-price timestamp. `RoycoDayKernel` uses the reported timestamp to enforce its staleness threshold, so a recent virtual-price update causes the stale-price check to pass even when the Chainlink leg has not been updated within the configured threshold.

This can cause deposits, redemptions, and accounting updates to use an outdated price for the CDO's underlying asset, potentially transferring value between users or tranches. The expected underlying asset is generally a stablecoin such as USDC, which limits the practical impact while its price remains close to the intended peg. However, the discrepancy can become material during a depeg or a feed outage, precisely when rejecting stale prices is most important.

Recommendations:

Preserve the timestamps of both components and return the older one, since the composed price is only as fresh as its oldest hop.

Customer Response:

Fixed in 353e926.

Fix Review:

Fix confirmed.

L-02: New losses inherit the original fixed-term expiry

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/accountant/RoycoDayAccountant.sol	Status: Acknowledged	

Description:

`RoycoDayAccountant` records junior tranche impermanent loss as JT absorbs collateral losses. While this balance remains outstanding, the fixed-term state blocks tranche operations and gives the underlying position time to recover, with future collateral appreciation first restoring JT. The first non-dust loss stamps `fixedTermEndTimestamp` as the current time plus `fixedTermDurationSeconds`.

However, `fixedTermEndTimestamp` is updated only when the market enters `FIXED_TERM` from `PERPETUAL`. Any additional loss observed while the market is already in `FIXED_TERM` increases `jtImpermanentLoss` without extending the deadline. The state transition is evaluated after the new loss has been applied, and an expired deadline forces the market back to `PERPETUAL` and clears the entire impermanent-loss balance.

This produces the following sequence:

1. An initial loss starts a fixed term ending at time `T`.
1. A material additional loss shortly before `T` increases JT's impermanent loss, but the deadline remains `T`.
1. At `T`, the next synchronization clears both the original balance and the newly incurred balance, even though the latter received almost no recovery period.

The edge case is more pronounced when no synchronization occurs between the new loss and expiry. The first synchronization after `T` first recognizes the collateral loss and adds it to `jtImpermanentLoss`, then immediately takes the expired-term branch and erases that same balance in the same transaction.

As a result, JT liquidity providers can permanently lose the recovery priority associated with a recent and potentially large loss. Subsequent collateral appreciation is treated as ordinary pro

rata gain instead of first restoring JT, and tranche operations resume immediately despite the fresh loss. This undermines the fixed term's intended protection and makes the recovery period depend on how close each loss occurs to the first loss's deadline.

Recommendations:

Extend the active fixed term whenever a material new impermanent loss is incurred, either by updating the PnL flow, or by creating a permissioned function that a guardian can call when it deems adequate.

Customer Response:

Acknowledged.

L-03: Hookless Balancer interactions leave fee-adjusted LPT NAV stale

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/kernels/base/liquidity-venue/balancer-v3/hooks/RoycoDayBalancerV3Hooks.sol	Status: Acknowledged	

Description:

Royco Day Balancer markets register their pools without hooks. Consequently, externally initiated swaps and liquidity operations do not synchronize tranche accounting or update the accountant's `lastLPTRawNAV` checkpoint.

A fee-generating swap can increase the pool's TVL per BPT while leaving `lastLPTRawNAV` at its pre-swap value. The checkpoint remains stale until a later Royco synchronization.

At that synchronization, `RoycoDayAccountant` first accrues the LPT yield share for the elapsed period using liquidity utilization derived from `lastSTEffectiveNAV` and the stale `lastLPTRawNAV`. Only after this accrual is processed does the kernel read and commit the pool's current LPT raw NAV. Because liquidity utilization is inversely proportional to LPT raw NAV, an unrecorded increase in BPT value makes the market appear more highly utilized than it was after the external operation.

Where the configured LPT yield model assigns a larger yield share at higher utilization, subsequent senior appreciation pays an excessive liquidity premium to LPT holders. Fee-generating pool activity can therefore cause part of the senior tranche's later appreciation to be transferred incorrectly from ST holders to LPT holders. The discrepancy grows with the unrecorded increase in BPT value, the time between the pool interaction and the next synchronization, and the utilization sensitivity of the LPT yield model.

Recommendations:

One option is to introduce the after-hooks to update the LPT raw NAV after each pool operation. Alternatively, consider acknowledging the issue and ensuring the markets have frequent synchronizations to mitigate the impact.

Customer Response:

Acknowledged. The protocol has indicated they will use dedicated keepers for each market to perform frequent synchronizations to mitigate this issue.

L-04: Makina recovery mode does not stop Royco from pricing DUSD

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/MakinaSharePriceOracle.sol	Status: Acknowledged	

Description:

Makina recovery mode is an emergency signal that disables normal Machine deposits and redemptions and restricts AUM updates to the Security Council. However, `convertToAssets` remains callable and calculates the share conversion from the Machine's last recorded AUM. The local `IMachine` interface does not expose `recoveryMode`, and `MakinaSharePriceOracle` reads the conversion without checking whether the Machine is in recovery mode:

```
  return
IMachine(MAKINA_MACHINE).convertToAssets(MACHINE_SHARES_TO_CONVERT_TO_ASSETS);
```

`MakinaSharePriceOracle` inherits `getPrice`, `poke`, and `previewPoke` from `ChainlinkPriceOracleBase`. Consequently, recovery mode does not make any of these calls revert. If the accounting-asset Chainlink feed remains valid, Royco treats the returned DUSD price as usable even when Makina has entered its emergency state and `convertToAssets` is still based on the last recorded AUM. The January 2026 DUSD incident demonstrates that recovery mode can be activated in response to a live exploit.

The issue can occur as follows:

1. Makina's Security Council enables recovery mode because the safety or value of DUSD is uncertain.

1. Makina blocks its own normal deposits and redemptions, but `convertToAssets` continues returning a value derived from the last recorded AUM.

1. Royco's oracle successfully returns that conversion combined with the Chainlink accounting-asset price, so Royco continues accepting, synchronizing, and valuing DUSD.

Royco can therefore increase its exposure to DUSD or process operations at an unreliable valuation precisely while Makina has signaled an emergency. If the cached AUM overstates the recoverable value of DUSD, deposits or redemptions executed at that price can shift the eventual loss among Royco tranches and existing tranche holders.

Recommendations:

Extend `IMachine` with the `recoveryMode` getter and make `MakinaSharePriceOracle` revert from its price path whenever recovery mode is enabled.

Alternatively, continuously monitor the Makina Machine's recovery mode status and immediately pause the affected Royco contract when recovery mode is enabled.

Customer Response:

Acknowledged.

L-05: Sync cadence changes impermanent-loss recovery priority

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/accountant/RoycoDayAccountant.sol	Status: Fixed	

Description:

`RoycoDayAccountant` tracks separate ST and JT raw NAVs. When JT covers an ST loss, ST retains its effective NAV through a cross-claim on JT's raw assets. However, the impermanent-loss ledger records only the nominal coverage transferred to ST and omits JT's own attributed loss. As a result, the amount recorded as recoverable JT impermanent loss is not uniquely determined by the final raw and effective NAVs.

For example, consider initial ST and JT raw and effective NAVs of 1,000 and 300:

1. If the shared asset price moves directly from 1.00 to 0.81, ST loses 190 and JT loses 57. JT covers the 190 ST loss, leaving effective NAVs of 1,000 and 53 and recording 190 of impermanent loss.

1. If a sync occurs at 0.90, the first leg records 100 of coverage and creates a 100 cross-claim on JT's remaining 270 raw NAV.

1. On the second move from 0.90 to 0.81, the cross-claim attributes 10 of JT's 27 raw-NAV loss to ST. The accountant therefore treats ST's second-leg loss as 100, records another 100 of coverage, and ends with 200 of impermanent loss.

Both paths end with identical raw NAVs of 810 and 243 and identical effective NAVs of 1,000 and 53, but their impermanent-loss ledgers differ by 10. Since this ledger has first claim on later appreciation, an identical recovery distributes value differently between ST and JT. Balancer pool operations can trigger accounting synchronizations, so a user can influence the number and placement of checkpoints during a decline and thereby change future recovery priority.

Consequently, tranche outcomes depend on user-influenceable accounting cadence rather than only on economic state. A JT participant can introduce intermediate checkpoints to increase the

recovery claim paid before ST recognizes yield, transferring later appreciation from ST holders to JT holders. The discrepancy can compound across multiple checkpoints and price movements.

Recommendations:

Given that it's confirmed that both JT and ST assets will be the same, use a single collateral NAV and apply each loss once through a junior-first waterfall, so splitting one price decline across multiple synchronizations cannot change the cumulative loss.

Customer Response:

Fixed in 7d8a023.

Fix Review:

Fix confirmed.

L-06: Whitelisted shareholders can bypass EntryPoint delays and oracle gating

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/kernels/base/RoycoDayKernel.sol	Status: Fixed	

Description:

When transfer-whitelist enforcement is enabled, `RoycoDayKernel` considers a recipient eligible to hold tranche shares when the recipient can call `deposit` on that tranche. The deployment template binds deposit and redeem to the same LP role, so every eligible holder can also invoke the synchronous tranche operations directly.

Direct operations bypass the EntryPoint's deposit and redemption delays, post-request oracle-update gate, and request-time value protections. However, the permissioned-market threat model treats whitelisted holders as trusted not to exploit pending oracle updates. An untrusted user therefore cannot reach this path, making the coupling a Low-severity defense-in-depth concern rather than an exploit within the intended threat model.

Recommendations:

For defense in depth, use separate roles for share-holder eligibility and operational authorization, granting direct deposit and redemption permission only to the `EntryPoint` and other explicitly trusted protocol components.
Alternatively, consider removing the whitelist check from the tranche transfer functions.

Customer Response:

Fixed in 931a6e7.

Fix Review:

Fix confirmed.

L-07: Executor controls the LPT redemption asset composition

Severity: Low	Impact: Low	Likelihood: Low
Files: src/entrypoint/RoycoDayEntryPoint.sol	Status: Fixed	

Description:

`RoycoDayEntryPoint` allows an LPT holder to queue shares for delayed redemption and authorize a third party to execute the request. However, the request does not record whether the holder prefers an in-kind or multi-asset redemption.

Instead, `executeRedemption` derives the route from the executor-controlled `sharesToRedeem` argument. A specific amount always executes in-kind, while `type(uint256).max` can select the multi-asset route when its redemption bound is wider.

A third-party executor can therefore select the output composition:

1. Passing `type(uint256).max` can redeem through the multi-asset path and send the pool's constituent collateral and quote assets to the receiver.
1. Passing a specific amount within the in-kind bound redeems in-kind and sends the LP token and any idle senior-share claim to the receiver.

The receiver can consequently receive a different asset basket than the user intended, or the request can be only partially filled. This does not directly cause a loss of funds, but it prevents users from controlling the form in which their position is redeemed.

Recommendations:

Add a redemption mode to `RedemptionRequest` and require the user to select it when creating the request. Execution should derive the route exclusively from this stored preference, while sharesToRedeem should control only the fill size.

Customer Response:

Fixed in 477bbe0.

Fix Review:

Fix confirmed.

L-08: Idle CDO composite prices cannot enforce source-specific staleness limits

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/IdleCDOTranchePriceOracle.sol src/kernels/base/RoycoDayKernel.sol	Status: Fixed	

Description:

`IdleCDOTranchePriceOracle` composes an Idle CDO tranche virtual price with a Chainlink-compatible feed. It returns the older of the two update timestamps:

```
    (price, updatedAt) = ChainlinkPriceOracleBase.getPrice();
    uint256 tranchePriceUpdatedAt = previewPoke();
    updatedAt = Math.min(updatedAt, tranchePriceUpdatedAt);
```

`RoycoDayKernel` checks this timestamp against one market-wide staleness threshold. Although taking the minimum ensures that both components satisfy this shared threshold, it cannot enforce distinct freshness requirements for the deviation-based Idle CDO clock and the Chainlink feed.

If the shared threshold is selected for the slower Idle CDO clock, Royco can accept a Chainlink answer older than the feed-specific limit. This can cause pricing operations to use an outdated underlying-asset price during a feed outage or depeg. Using the stricter threshold for both components avoids this risk but may unnecessarily halt the market.

Recommendations:

Move staleness validation into `IdleCDOTranchePriceOracle` and configure separate thresholds for the Chainlink feed and the Idle CDO virtual-price clock. `getPrice()` should revert when either source exceeds its own threshold, while continuing to return the older of the two timestamps so the execution gate advances only after both components have updated.

Customer Response:

Fixed in f8e4d8d.

Fix Review:

Fix confirmed.

L-09: Balancer exit fee after hook-triggered reinvestment

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/kernels/base/liquidity-venue/balancer-v3/hooks/RoycoDayBalancerV3Hooks.sol src/libraries/logic/FeeAndLiquidityPremiumLogic.sol src/libraries/logic/liquidity-venue/BalancerV3VenueLogic.sol lib/balancer-v3-monorepo/pkg/vault/contracts/Vault.sol	Status: Fixed	

Description:

Balancer V3 records whether liquidity has been added to a pool during the current outermost Vault unlock session. If a proportional removal from the same pool occurs later in that session, Balancer reduces every token output by the pool's static swap fee. This round-trip protection prevents a user from temporarily adding liquidity, interacting with the pool, and immediately withdrawing the liquidity again.

An external BPT holder can unintentionally trigger this protection while performing an otherwise ordinary proportional withdrawal:

1. The holder initiates a proportional BPT removal through a Balancer router, which opens a Vault `unlock` session.
2. Before processing the removal, the Vault calls Royco's `onBeforeRemoveLiquidity` hook.
3. Because the removal was not initiated by the Royco kernel, the hook calls `syncTrancheAccounting()`.
4. If this synchronization mints a nonzero liquidity-provider-tranche premium, it attempts to reinvest the premium's senior-tranche shares into the same pool.

5. Reinvestment calls `vault.unlock` again. Since the Vault is already unlocked, the nested call remains in the holder's existing session rather than creating a new session.

6. If the reinvestment add succeeds, it sets Balancer's add-liquidity flag for the pool in that session.

7. The holder's proportional removal then resumes. Balancer observes the flag and charges the round-trip fee, reducing every proportional token output by the pool's static swap fee.

The BPT holder did not add liquidity or perform an add/remove round trip. Royco's internal accounting operation is nevertheless attributed to the holder's session. If the holder supplied minimum outputs based on a normal proportional withdrawal, the unexpected reduction causes the transaction to revert; with looser minimums, the holder receives less than expected.

Recommendations:

Move premium reinvestment from the pre-operation accounting sync to the post-operation flow. Hook-triggered syncs should only stage premium shares, preventing them from adding liquidity during an external holder's Vault session.

Customer Response:

Fixed in 7e4a754.

Fix Review:

Fix confirmed.

L-10: The liquidation threshold cannot be updated after the junior buffer is exhausted

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/libraries/logic/UtilizationLogic.sol src/accountant/RoycoDayAccountant.sol src/libraries/logic/DepositLogic.sol	Status: Fixed	

Description:

When collateral NAV and minimum coverage are nonzero but junior effective NAV is zero, `_computeCoverageUtilization` returns `type(uint256).max` to represent effectively infinite coverage utilization.

In the affected version, `withSyncedAccounting` accepted a liquidation-threshold change only when either the threshold was unchanged or the new threshold was strictly greater than post-operation utilization:

```
preThreshold == postThreshold || postThreshold > postUtilization
```

`setLiquidationCoverageUtilization` is the only post-initialization setter for the threshold. Therefore, every genuine update makes the equality false. Because no `uint256` value is greater than `type(uint256).max`, the second condition is also impossible to satisfy. Once the junior buffer is exhausted, the setter consequently accepts only a no-op rewrite of the existing threshold.

Setters that do not modify the threshold are not rejected by this particular condition, although they remain subject to the modifier's other safety checks. For example, setting minimum coverage to zero reduces coverage utilization to zero and can succeed. This is not a reversible workaround: restoring any positive minimum coverage makes utilization jump from zero to

`type(uint256).max`, failing the separate requirement that utilization remain at or below WAD or not increase.

Recommendations:

Allow non-decreasing liquidation thresholds even when utilization is saturated. Replace the equality check with `preThreshold <= postThreshold`, while retaining the existing requirement that a decreased threshold must remain above current utilization. Keep the saturation sentinel unchanged.

Customer Response:

Fixed in d0751ab.

Fix Review:

Fix confirmed.

L-11: Multi-asset flows reject partial liquidity healing

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/libraries/logic/AccountingSyncLogic.sol	Status: Acknowledged	

Description:

Multi-asset deposits with a collateral leg and multi-asset redemptions contain an initial leg that can temporarily worsen liquidity utilization. Instead of reverting immediately, `postOpSyncTrancheAccounting` records a pending violation that the final improving leg clears only if utilization finishes at or below 100%.

This does not account for a flow that begins while liquidity utilization is already above 100% and improves it without fully resolving the breach. For example:

1. A market enters a multi-asset flow with liquidity utilization at 200%.
1. The initial senior deposit or liquidity provider redemption leg leaves utilization above 100%, which sets `PENDING_LIQUIDITY_VIOLATION`.
1. The final liquidity provider deposit or senior redemption improves utilization to 150%.
1. Since 150% remains above 100%, the final leg does not clear the pending flag, and `_exitMultiAssetFlow` reverts the entire flow.

The flag does not preserve utilization at flow entry, so the exit logic cannot distinguish a net-worsening flow from a recovery action that improves an existing breach. Standalone improving operations, in contrast, can partially heal a breached state.

As a result, beneficial multi-asset recapitalization or deleveraging actions revert unless one flow completely restores the liquidity requirement, which can delay recovery of an unhealthy market.

Recommendations:

Consider if it's worth it to introduce more logic to mitigate this edge case.

One option is to cache liquidity utilization at flow entry and permit the flow when final utilization is at most 100% or strictly below the entry value.

Alternatively, consider acknowledging and documenting this issue.

Customer Response:

Acknowledged.

L-12: Chainlink updates can mask stale ERC4626 share prices

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/ERC4626SharePriceOracle.sol	Status: Fixed	

Description:

`ERC4626SharePriceOracle` composes the vault conversion returned by `convertToAssets` with a Chainlink price. However, the oracle inherits directly from `ChainlinkPriceOracleBase` and reports only the Chainlink `updatedAt` value, even though the composed price also depends on the vault share price.

ERC-4626 does not require `convertToAssets` to accrue continuously or expose when its accounting was last updated. A discretely accounted vault could therefore retain an old conversion value while a new Chainlink report advanced the composed oracle timestamp. The entry point could then consider the oracle updated after a queued request even though the vault component used for pricing had not changed since before the request.

The only `ERC4626SharePriceOracle` integration currently present is sNUSD, whose conversion changes continuously during reward vesting, so the issue is not exploitable in the existing deployment. However, reusing this adapter for a vault with periodic accounting would allow a Chainlink heartbeat alone to satisfy the execution gate with stale share-price information.

Recommendations:

Derive a separate vault share-price clock from observed `convertToAssets` deviations through `OracleClockBase`, and have `ERC4626SharePriceOracle` report the minimum of that checkpoint timestamp and the Chainlink timestamp.

Customer Response:

Fixed in a9c43ed.

Fix Review:

Fix confirmed.

L-13: Deprecated answeredInRound check limits oracle compatibility

Severity: Low	Impact: Low	Likelihood: Low
Files: src/oracle/base/ChainlinkPriceOracleBase.sol	Status: Fixed	

Description:

`ChainlinkPriceOracleBase` is designed to consume both Chainlink and Chainlink-compatible oracles. Its `getPrice` function reads the latest round and rejects the result unless `answeredInRound` is greater than or equal to `roundId`:

```
require(answeredInRound >= roundId, INCOMPLETE_PRICE());
```

However, Chainlink documents `answeredInRound` as deprecated because it was only needed when an answer could take multiple rounds to compute. It is therefore no longer a meaningful completeness signal for current Chainlink feeds, and the check provides no additional protection for those feeds.

Moreover, an AggregatorV3-compatible oracle may return a placeholder value for the deprecated field while still returning a positive, recently updated price. In that case, `getPrice` reverts with `INCOMPLETE_PRICE` even though the price passes the relevant validity and freshness checks. Since all oracle consumers depend on `getPrice`, configuring such a feed would prevent the protocol from performing price-dependent operations until the oracle is replaced or the implementation is upgraded.

Recommendations:

Ignore the deprecated `answeredInRound` return value and remove the comparison against `roundId`.

Customer Response:

Fixed in f10d768.

Fix Review:

Fix confirmed.

L-14: previewRedeem might be unsafe for generalized ERC4626 pricing

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/ERC4626SharePriceOracle.sol	Status: Acknowledged	

Description:

`ERC4626SharePriceOracle` supports a `PREVIEW_REDEEM` mode intended to include exit fees and haircuts in the reported collateral price. To obtain a WAD-scaled result directly, it calls `previewRedeem` with `10 ** (18 + shareDecimals - assetDecimals)` raw shares. For an 18-decimal share over a 6-decimal asset, this is `1e30` raw shares, or `1e12` whole shares. This amount can greatly exceed the vault's total supply or available liquidity, causing an integration-specific revert or returning a nonlinear redemption quote that is not representative of one share or Royco's actual position.

More generally, `previewRedeem` is an amount-specific, transaction-specific quote based on current on-chain conditions. The ERC-4626 specification explicitly warns that preview methods are manipulable and are not always safe as price oracles. Unlike `convertToAssets`, `previewRedeem` also includes withdrawal fees, slippage, and other conditions that may change without any change in the vault's underlying assets.

Royco multiplies its entire collateral balance by this quote and treats any change as collateral PnL during the next accounting sync. If the quoted haircut can be manipulated, an attack can proceed as follows:

1. The attacker temporarily increases the haircut or otherwise depresses `previewRedeem`.

1. An accounting sync observes the lower quote and realizes the change as a collateral loss, which is absorbed by JT first.

1. The attacker restores the quote after the loss has changed the market state or erased JT's recovery claim.

1. A later sync treats the restoration as collateral appreciation, potentially allocating it to different holders and charging premiums or protocol fees on artificial PnL.

Even a legitimate governance change to a fixed exit fee creates the same accounting mismatch: the fee change is immediately recorded as economic PnL despite no corresponding gain or loss in underlying assets.

Recommendations:

It's recommended to choose carefully which vaults can be priced using `previewRedeem` instead of `convertToAssets`.

Customer Response:

Acknowledged, the `previewRedeem` pricing is intended to be used for the Royco Dawn vaults by now, which are a safe integration.

Informational Severity Issues

I-01: Risk parameters cannot be changed while the market is paused

Files:

src/accountant/RoycoDayAccountant.sol
src/kernels/base/RoycoDayKernel.sol

Description:

The `withSyncedAccounting` modifier synchronizes kernel accounting before and after applying a parameter change. Both synchronizations call `syncTrancheAccountingFromAccountant`, which is guarded by `whenNotPaused`. Consequently, while the kernel is paused, every setter using `withSyncedAccounting` reverts before its parameter change can take effect.

The affected functions are the four protocol-fee setters, `setMinCoverage`, `setLiquidationCoverageUtilization`, `setMinLiquidity`, `setMaxYieldShares`, `setFixedTermDuration`, and `setDustTolerance`. The two YDM setters are not affected because they use a best-effort synchronization whose failure is tolerated.

If incident mitigation requires changing one of these parameters, governance must temporarily unpause the kernel. In the canonical deployment, the root multisig can atomically batch an unpause, parameter update, and subsequent pause, avoiding an inter-transaction exposure window. Nevertheless, this procedure temporarily disables the pause guard while the synchronization executes and becomes more fragile if pause and parameter-management responsibilities are assigned to different executors.

Recommendations:

Give the modifier a path that works while paused or change it.

Customer Response:

Acknowledged.

I-02: In-kind liquidity redemptions revert for holders without the senior role

Files:

src/libraries/logic/TrancheClaimsLogic.sol
src/libraries/logic/RedemptionLogic.sol
src/kernels/base/RoycoDayKernel.sol
src/factory/templates/RoycoDayBalancerV3MarketDeploymentTemplate.sol

Description:

On a market configured to enforce the tranche-share transfer whitelist, an in-kind liquidity-tranche redemption can revert when its receiver is not authorized to deposit into the senior tranche.

A liquidity-tranche redemption distributes the redeemer's proportional claims on both the venue LP token and any senior-tranche shares held idle as liquidity premium. Before calculating those claims, the redemption synchronizes accounting, mints newly accrued premium shares, and attempts to reinvest the idle senior-share balance. If reinvestment does not execute and the redeemer's proportional idle-share claim rounds to a nonzero amount, the redemption transfers those senior shares directly from the kernel to the receiver.

That transfer invokes the senior tranche's balance-update hook. On a whitelist-enforcing market, the hook asks the access manager whether the receiver can call deposit on the senior tranche. The deployment template binds that selector to `ST_LP_ROLE`, while liquidity-tranche redemption is independently bound to `LT_LP_ROLE`. Consequently, a caller can be authorized to redeem liquidity shares while the selected receiver is not authorized to receive the senior-share portion, causing the transaction to revert with `ACCOUNT_NOT_WHITELISTED_TRANCHE_LP`.

The roles are not automatically paired. Granting both roles to a liquidity provider makes self-receiving redemptions work, but does not guarantee redemptions sent to another receiver. The zero address, kernel, protocol fee recipient, and Balancer vault were exempt from the old transfer check, although the zero address could not be supplied as a redemption receiver.

This behavior is consistent with enforcing senior-tranche eligibility on every senior-share recipient, but creates an undocumented cross-tranche dependency for in-kind liquidity exits. The multi-asset route avoids transferring senior shares by redeeming them to collateral, but it is

not an unconditional fallback: it is disabled during fixed term and depends on venue execution, slippage bounds, and the final accounting requirements.

Recommendations:

Consider removing the whitelist check from the transfer functions in tranche shares.

Alternatively, consider acknowledging and documenting this issue.

Customer Response:

Fixed in 931a6e7.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.